

נ.נ. בתכנון משולב חומרה/תוכנה

אביב – 00460882

תרגול 1 - Intro to perf

אוהד איתן

Administrative

Grades:

- HW1 (10%)
- HW2 (15%)
- Final Project (75%)

Technical

Moodle

Slides would be available before the class

- You are allowed to use AI.
- However, it is up to you to understand the subject very well

Technical

- Moodle
- Slides would be available before the class
- You are ~~allowed~~ **encouraged** to use AI.
- However, it is up to you to understand the subject very well

Assistance

By email (personal)

Moodle Forum (public)

Last year there were around 24 students...

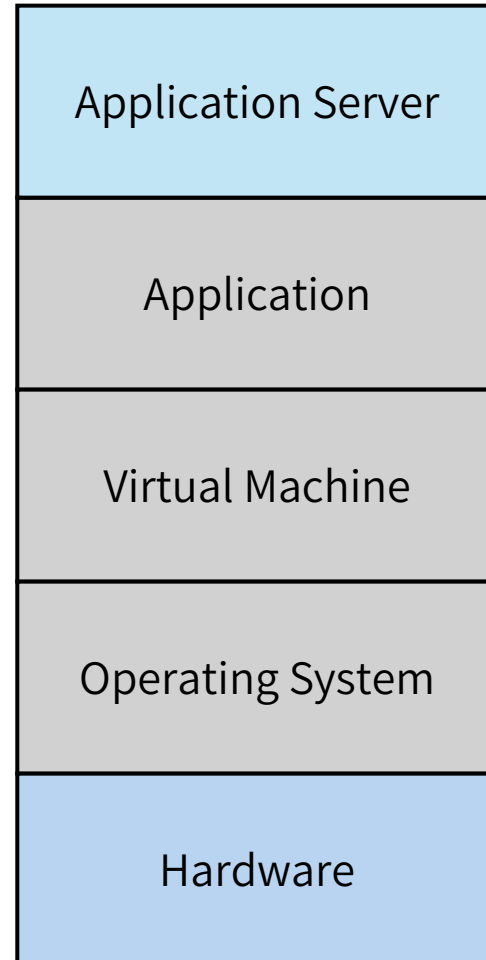
My Motivation

- Profiling tools: Valgrind, gprof, htop

```
pen@kali:~/...$ valgrind --tool=callgrind --dump-instrns=yes ./...
Samples: 226K of event 'cycles:P', Event count (approx.): 182364861302
Overhead Command Shared Object Symbol
21.36% run libcrypto++.so.8.9.0 [.] CryptoPP::KeccakF1600(unsigned long*)
7.08% run libc.so.6 [.] _int_malloc
6.10% run [kernel.kallsyms] [k] 0xfffffffff9579e73d
3.37% run libc.so.6 [.] __memmove_avx_unaligned_erms
1.66% run [kernel.kallsyms] [k] 0xfffffffff967b17a7
```

```
Ir file:function
-----
221,320,718,800 (55.63%) ???::CryptoPP::KeccakF1600(unsigned long*) [/usr/lib/x86_64-linux-gnu/libcrypto++.so.8.9.0]
65,912,134,500 (16.57%) ./string/./sysdeps/x86_64/multiarch/memmove-vec-unaligned-erms.S: __memcpy_avx_unaligned_erms [/usr/lib/x86_64-linux-gnu/libc.so.6]
11,968,827,500 ( 3.01%) ???::0x000000000001a1020 [/usr/lib/x86_64-linux-gnu/libcrypto++.so.8.9.0]
11,763,454,526 ( 2.96%) ./malloc/./malloc/malloc.c: int malloc [/usr/lib/x86_64-linux-gnu/libc.so.6]
```

Motivation



Disclaimer

Software and Hardware are complex (solve complex problems).

Software-Hardware are difficult to comprehend.

Application usage scenario is hard to predict.

Introduction to Profiling

Profiling is the first step in optimizing and tuning your application.

Profiling an application helps you understand where most of the execution time is spent.

Profiling enables you to find hotspots and bottlenecks in your application so you can decide where to focus your optimization efforts.

Qualities of a great profiler

Accurate

Low-overhead

Fast convergence

Flexible

Portable

Transparent

Low storage overhead

Perf

perf (Performance Counters for Linux) is a powerful profiling and performance analysis tool included in the Linux kernel.

It allows developers and system administrators to measure and analyze various performance metrics of applications and the entire system.

With perf, you can:

Analyze CPU usage and detect bottlenecks.

Profile functions and identify hotspots in programs.

Measure cache misses, branch mispredictions, and other low-level performance metrics.

Trace system calls and context switches.

The Power of perf

Built-in Kernel Support: No need for external installations.

Low Overhead: Minimal impact on system performance.

Comprehensive Data: Provides both hardware and software performance metrics.

Versatility: Can be used for high-level system monitoring or low-level CPU analysis.

Intro to perf

Checking if perf is installed

perf --version

Intro to perf

Installing perf:

```
sudo apt install linux-tools-common linux-tools-$(uname -r)
```

If everything is right:

```
ohade@ohade-VirtualBox:~/Desktop$ perf --version  
perf version 6.11.11
```

The perf tool offers a rich set of commands to collect and analyze performance and trace data. The most common ones are: stat, record, report.

stat Run a command and gather performance counter statistics

record Run a command and record its profile into perf.data

report Read perf.data (created by perf record) and display the profile

Events

The perf tool supports a list of measurable events, coming from different sources.

For instance, some event are pure kernel counters, in this case they are called **software events**. Examples include: context-switches, minor-faults.

Another source of events is the processor itself and its Performance Monitoring Unit (PMU).

It provides a list of events to measure micro-architectural events such as the number of cycles, instructions retired, L1 cache misses and so on.

Those events are called **PMU hardware events** or **hardware events**.

• cpu-cycles OR cycles	[Hardware event]
• instructions	[Hardware event]
• cache-misses	[Hardware event]
• branch-instructions OR branches	[Hardware event]
• branch-misses	[Hardware event]
• cpu-clock	[Software event]
• page-faults OR faults	[Software event]
• minor-faults	[Software event]
• major-faults	[Software event]
• context-switches OR cs	[Software event]
• cpu-migrations OR migrations	[Software event]

For any of the supported events, perf can keep a running count during process execution.

In counting modes, the occurrences of events are simply aggregated and presented on standard output at the end of an application run.

To generate these statistics, use the stat command of perf. For instance:

```
perf stat -B dd if=/dev/zero of=/dev/null count=1000000

1000000+0 records in
1000000+0 records out
512000000 bytes (512 MB) copied, 0.956217 s, 535 MB/s

Performance counter stats for 'dd if=/dev/zero of=/dev/null count=1000000':

          5,099 cache-misses          #    0.005 M/sec (scaled from 66.58%)
       235,384 cache-references       #    0.246 M/sec (scaled from 66.56%)
     9,281,660 branch-misses          #    3.858 %    (scaled from 33.50%)
  240,609,766 branches                #  251.559 M/sec (scaled from 33.66%)
 1,403,561,257 instructions            #    0.679 IPC  (scaled from 50.23%)
 2,066,201,729 cycles                 #  2160.227 M/sec (scaled from 66.67%)
         217 page-faults             #    0.000 M/sec
           3 CPU-migrations           #    0.000 M/sec
          83 context-switches         #    0.000 M/sec
   956.474238 task-clock-msecs        #    0.999 CPUs

0.957617512 seconds time elapsed
```

The perf tool can be used to count events on a:

per-thread

per-process

per-cpu

In per-thread mode, the counter only monitors the execution of a designated thread.

When the thread is scheduled out, monitoring stops.

When a thread migrated from one processor to another, counters are saved on the current processor and are restored on the new one.

Repeated measurement

It is possible to use perf stat to run the same test workload multiple times and get for each count, the standard deviation from the mean.

```
perf stat -r 5 sleep 1

Performance counter stats for 'sleep 1' (5 runs):

<not counted> cache-misses
      20,676 cache-references      #    13.046 M/sec    ( +-   0.658% )
       6,229 branch-misses        #     0.000 %       ( +-  40.825% )
<not counted> branches
<not counted> instructions
<not counted> cycles
      144 page-faults             #     0.091 M/sec    ( +-   0.139% )
         0 CPU-migrations          #     0.000 M/sec    ( +-   -nan% )
         1 context-switches        #     0.001 M/sec    ( +-   0.000% )
    1.584872 task-clock-msecs      #     0.002 CPUs     ( +-  12.480% )

1.002251432 seconds time elapsed ( +-   0.025% )
```

Here, sleep is run 5 times and the mean count for each event, along with ratio of std-dev/mean is printed.

Sampling with perf record

There are several commands associated with sampling: record, report, annotate.

You must first collect the samples using perf record.

This generates an output file called perf.data.

That file can then be analyzed, possibly on another machine, using the perf report and perf annotate commands.

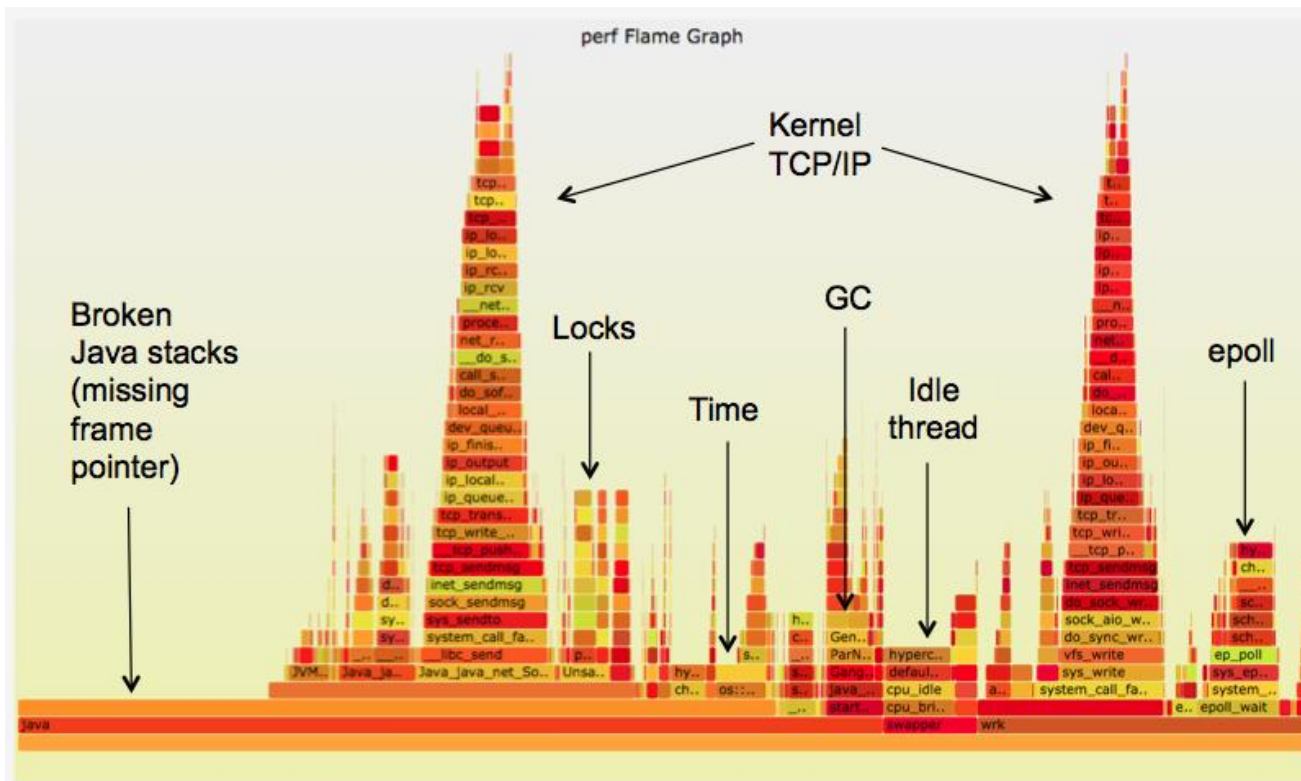
```
perf record ./noploop 1
```

```
[ perf record: Woken up 1 times to write data ]
```

```
[ perf record: Captured and wrote 0.002 MB perf.data (~89 samples) ]
```

Flame Graph

FlameGraphs are a popular way to visualize stack traces and break down execution time. The perf tool supports natively generating flame graphs using the perf script report flamegraph command.



```
perf record -a -g -F 99 sleep 60
perf script report flamegraph
google-chrome flamegraph.html
```

Callchains

The overhead can be shown in two columns as 'Children' and 'Self' when perf collects callchains.

The 'self' overhead is simply calculated by adding all period values of the entry - usually a function (symbol).

The sum of all the 'self' overhead values should be 100%.

The 'children' overhead is calculated by adding all period values of the child functions so that it can show the total overhead of the higher level functions even if they don't directly execute much.

'Children' here means functions that are called from another (parent) function.


```
void foo(void) {  
    /* do something */  
}  
  
void bar(void) {  
    /* do something */  
    foo();  
}  
  
int main(void) {  
    bar()  
    return 0;  
}
```

In this case 'foo' is a child of 'bar', and 'bar' is an immediate child of 'main' so 'foo' also is a child of 'main'. In other words, 'main' is a parent of 'foo' and 'bar', and 'bar' is a parent of 'foo'.

Suppose all samples are recorded in 'foo' and 'bar' only. When it's recorded with callchains the output will show something like below in the usual (self-overhead-only) output of perf report:

```
Overhead  Symbol
.....
 60.00%   foo
          |
          --- foo
              bar
              main
              __libc_start_main

 40.00%   bar
          |
          --- bar
              main
              __libc_start_main
```

Children	Self	Symbol
.....		
100.00%	0.00%	__libc_start_main
	---	__libc_start_main
100.00%	0.00%	main
	---	main
		__libc_start_main
100.00%	40.00%	bar
	---	bar
		main
		__libc_start_main
60.00%	60.00%	foo
	---	foo
		bar
		main
		__libc_start_main

Acknowledge

- Some Ideas and slides, were taken from the internet, special thanks to:

Suyog K Sethia

<https://www.brendangregg.com/perf.html>

<https://perfwiki.github.io/main/tutorial/>